

Controlling the mutation in Large Language Models for the efficient evolution of algorithms

Yin, H.; Kononova, A.V.; Bäck, T.H.W.; Stein, N. van; García-Sánchez, P.; Hart, E.; Thomson, S.L.

Citation
Yin, H., Kononova, A. V., Bäck, T. H. W., & Stein, N. van. (2025). Controlling the mutation in Large Language Models for the efficient evolution of algorithms. *Lecture Notes In Computer Science*, 403-417. doi:10.1007/978-3-031-90065-5_25

Version:	Publisher's Version
License:	Licensed under Article 25fa Copyright Act/Law (Amendment Taverne)
Downloaded from:	https://hdl.handle.net/1887/4295598

Note: To cite this publication please use the final published version (if applicable).

Controlling the Mutation in Large Language Models for the Efficient Evolution of Algorithms

Haoran Yin(✉), Anna V. Kononova, Thomas Bäck, and Niki van Stein

LIACS, Leiden University, Leiden, The Netherlands
h.yin@liacs.leidenuniv.nl

Abstract. The integration of Large Language Models (LLMs) with evolutionary computation has introduced a promising paradigm for automating the design of metaheuristic algorithms. However, existing frameworks, such as the Large Language Model Evolutionary Algorithm (LLaMEA), often lack precise control over mutation mechanisms, leading to inefficiencies in solution space exploration and potentially suboptimal convergence. This paper introduces a novel approach to mutation control within LLM-driven evolutionary frameworks, inspired by theory of genetic algorithms. Specifically, we propose dynamic mutation prompts that adaptively regulate mutation rates, leveraging a heavy-tailed power-law distribution to balance exploration and exploitation. Experiments using GPT-3.5-turbo and GPT-4o models demonstrate that the former fails to adhere to the specific mutation instructions. At the same time, the latter is able to control its mutation rate under the instruction of well-constructed mutation prompts. Further experiments show that the introduction of these dynamic rates can improve the convergence speed and adaptability of LLaMEA when using GPT-4o. This work sets a starting point for better controlled LLM-based mutations in code optimization tasks, paving the way for further advancements in automated metaheuristic design.

Keywords: Metaheuristic Algorithms · Algorithm Evolution · Mutation Control · Large Language Models · Evolutionary Computation

1 Introduction

The development of metaheuristic algorithms [3] has been foundational for tackling complex optimization problems across domains such as engineering [22], artificial intelligence [16], and computational biology [17]. Traditionally, these algorithms have relied on expert-crafted designs and manual tuning, a process that, while effective, is labor-intensive and constrained in scalability. Recent advances in Large Language Models (LLMs) [6,23] have brought new opportunities to this field, enabling automated generation and refinement of metaheuristics through natural language processing capabilities. The synergy between LLMs and evolutionary computation (EC) [21] represents a transformative shift, leveraging

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
P. García-Sánchez et al. (Eds.): EvoApplications 2025, LNCS 15613, pp. 403–417, 2025.
https://doi.org/10.1007/978-3-031-90065-5_25

LLMs for generating and enhancing code, while EC methods provide global search and iterative improvement capabilities.

A variety of methodologies exemplify this integration. The Large Language Model Evolutionary Algorithm (LLaMEA) [18] employs LLMs to iteratively generate, mutate, and select optimization algorithms, outperforming state-of-the-art methods in benchmark tasks. Similarly, the Algorithm Evolution using Large Language Models (AEL) framework [13] and the Evolution of Heuristics (EoH) approach [14] demonstrates how LLMs can autonomously evolve heuristics using in-context learning and feedback loops. Further, approaches such as Evolutionary Optimization with LLMs (EvoLLM) [12] show how prompt engineering and mutation operators enhance LLMs' ability to navigate search spaces effectively. These studies illustrate the potential for LLM-driven algorithmic frameworks to automate tasks traditionally requiring significant domain expertise.

Despite these advancements, existing frameworks often exhibit inefficiencies in exploring the solution space. A key limitation lies in the lack of precise control over mutation mechanisms during algorithm evolution, which can lead to redundant evaluations and suboptimal convergence. Addressing this gap, this paper introduces a novel mutation control mechanism for LLM-driven evolutionary frameworks. Drawing inspiration from genetic algorithms where a mutation rate sampled from a heavy-tailed power-law distribution is most beneficial [7], we propose dynamic mutation prompts that adaptively modulate the mutation rate to balance exploration and exploitation, aiming to enhance the convergence speed of LLaMEA.

The remainder of this paper is structured as follows: Sect. 2 reviews the state-of-the-art in LLM-EC integration. Section 3 details our proposed mutation control mechanism and its integration into the LLaMEA framework. Sections 4 and 5 describes the experimental setup and results, and Sect. 6 discusses the implications and future directions of our work.

2 Related Work

The generation and optimization of metaheuristic algorithms using LLMs has become a research hotspot in the field of research that explores the integration of large-scale language models with evolutionary computation, including EoH and LLaMEA [14,18,19,21]. LLaMEA is the basis of our work as it provides an open-source modular framework that we can easily expand on. It is designed to utilize LLMs to automatically generate and improve optimization algorithms. The core idea is to achieve steady improvement of algorithm performance through an automated evolutionary process driven by language models, and an evolutionary strategy (1, 1) or (1 + 1), including algorithm generation, mutation and selection [4]. Specifically, the LLaMEA process consists of the following steps:

– **Generation:** The LLM generates the algorithms based on the generation prompt. This prompt can include problem descriptions, constraints, and a example of python code structure for algorithms. By adjusting the prompt, the model can be guided to generate more targeted algorithmic candidates.

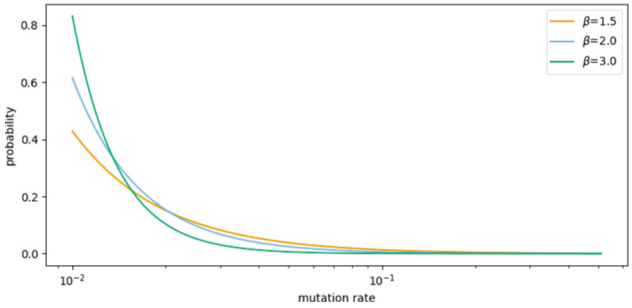


Fig. 1. The distribution of Eq. 1. The areas under the curves are all 1.

- **Mutation:** When the LLM is tasked with generating a new algorithm, they receive not only the initial generation prompt, but also mutation prompts to make the LLM mutate algorithms. These mutation prompts guide the model to either make slight refinements to the existing algorithm or completely redesign it. The latter approach is analogue to a restart in Evolutionary Algorithm optimization, thereby enhancing the model's exploration capabilities.
- **Selection:** LLaMEA evaluates the best current and newly generated algorithms based on performance metrics. Only the algorithm that excels in multidimensional metrics is allowed to proceed to the next iteration. This performance feedback-based selection mechanism enables LLaMEA to continuously optimize the quality of the generated algorithms.

Through this iterative process, LLaMEA enables the algorithms to evolve, demonstrating greater efficiency and flexibility in solving complex problems [18]. This framework not only reduces development time, but also generates algorithmic solutions with high performance on specific tasks, which has strong potential for practical applications. However, research in LLaMEA so far has mainly focused on the automatic generation of algorithms, while careful control of the mutation behavior during their evolution has been neglected, which makes the results uncontrollable and leads to a potential waste of computational resources.

To explore the potential improvement of the mutation mechanism in LLaMEA, the choice of mutation rate is crucial. Traditional genetic algorithms recommend a fixed mutation rate, which is generally effective enough in simple unimodal optimization problems [3,10]. However, for complex and multimodal optimization problems, dynamically adjusting the mutation rate has been shown to provide significant performance gains [5,11]. Therefore, we propose to use the fast mutation operator $fmut_\beta$ by Doerr et al. [7], to help improve the convergence speed of LLaMEA. $fmut_\beta$ is a heavy-tailed mutation operator for mutation steps, which is specifically beneficial for dealing with multimodal functions, where static mutation rates may not be efficient [2,9]. It employs a randomized mutation rate α/n, where α is an integer that follows a power-law distribution that does not concentrate strongly around its mean, and n represents the dimension of the problem. This allows for a greater range of mutation strengths,

potentially improving the algorithm's ability to escape local optima and explore the fitness landscape more effectively. The power-law probability distribution is as illustrated in Fig. 1 and defined as follows:

$$Pr(\alpha/n) = \left(\sum_{i=1}^{\lfloor n/2 \rfloor} i^{-\beta} \right)^{-1} \alpha^{-\beta} \tag{1}$$

Studies show that using a heavy-tailed mutation rate significantly reduces the optimization time for complex problems compared to a fixed mutation rate, and this approach has been proven to optimize functions more efficiently, particularly for complex problems with multiple local optima [7].

3 LLaMEA with Dynamic Mutation Rate

By integrating dynamic mutation rates from discrete power-law distributions into the LLaMEA framework, we can adapt a concept that has proven to be very useful in genetic algorithms for pseudo-Boolean optimization problems to our domain of LLMs for code generation.

The effort of introducing a dynamic mutation rate to LLaMEA starts with the refinement of the existing mutation prompt. In the LLaMEA framework, each generated algorithm is produced by an LLM using the parent algorithm and a mutation prompt, but this mutation prompt is as simple as 'Either refine or redesign to improve the algorithm'. To make this prompt more specific, we modified it to include the information about the mutation rate expected from the LLM. Therefore, the question of *whether LLMs can correctly understand and accurately execute the mutation prompt* needs to be investigated. Once this is established, the mutation rate should be probabilistic and follow a given distribution, inspired by $fmut_\beta$.

We start with experiments to establish whether LLMs can achieve a fixed mutation rate when given a mutation prompt. We measure the code mutation effect by first computing the *logarithm of the ratio* of the fixed requested mutation rate x (between 0 and 1) to that of the actual mutation rate x_i delivered by LLaMEA, where i denotes different generated algorithms. Then calculating the mean squared error (MSE) as shown in Eq. 2:

$$MSE(prompt, x) = \frac{\sum_{i=1}^{N} \log^2 \frac{x_i}{x}}{N} \tag{2}$$

where N is the number of generated algorithm codes. We use $\log \frac{x_i}{x}$ instead of $x_i - x$ commonly used in MSE formulas to ensure a fair measure of the difference between the requested code difference and delivered code difference, e.g., when the requested code difference is 0.1 the delivered code difference of 0.01 should be considered as bad as 1.0, not as bad as 0.19.

To account for the distribution of mutation rates, we define target-distribution-weighted score (TDW-score) to measure the overall effect of different prompts as shown in Eq. 3:

$$\text{TDW-score}(prompt) = \frac{\sum_{j=1}^{M} w_j MSE(prompt, x^j)}{M} \tag{3}$$

where x^j is the element of a set of fixed mutation rates with which we experiment, w_j is the weight of $MSE(prompt, x^j)$ and M is the number of experimented fixed mutation rates. With the MSE and the TDW-score, we can visualize the difference between the effects of different investigated prompts more intuitively.

We can then explore the *impact of mutation prompts on the LLaMEA generation algorithm when combined with the dynamic mutation rate*. In Eq. 1, we set n to be the number of lines of the parent algorithm code, and use the mutation rate obtained by sampling from Eq. 1 as the value of x in the mutation prompts, and we use the same performance metric as LLaMEA [18], *Area Over the Convergence Curve* (AOCC), as introduced in [20].

3.1 Manually Constructed Prompts

To explore how to make mutation prompts effective, we start with the most basic and intuitive **prompt 1**, which contains only a description of the mutation task and a value of the mutation rate. Then, step by step, we continue to add new content, including mandatory clarification (**prompt 2**), strict limitations (**prompt 3**), example specification (**prompt 4**), and context-specific requirements (**prompt 5**). These prompts are shown below:

Manual prompts

Prompt 1 Now, refine the strategy of the selected solution to improve it. Make sure that you only change x% of the code.

Prompt 2 Now, refine the strategy of the selected solution to improve it. Make sure that you only change x% of the code. This changing rate x% is the mandatory requirement.

Prompt 3 Now, refine the strategy of the selected solution to improve it. Make sure that you only change x% of the code. This changing rate x% is the mandatory requirement, you cannot change more or less than this rate.

Prompt 4 Now, refine the strategy of the selected solution to improve it. Make sure that you only change x% of the code, which means if the code has 100 lines, you can only change x lines, and the rest lines should remain the same. This changing rate x% is the mandatory requirement, you cannot change more or less than this rate.

Prompt 5 Now, refine the strategy of the selected solution to improve it. Make sure that you only change x% of the code, which means if the code has 100 lines, you can only change x lines, and the rest lines

should remain the same. For this code, it has n lines, so you can only change $\lfloor n \times x/100 \rfloor$ lines, the rest $n - \lfloor n \times x/100 \rfloor$ lines should remain the same. This changing rate $x\%$ is the mandatory requirement, you cannot change more or less than this rate.

3.2 Automatically Generated Prompts

Popular research at the forefront has shown that LLMs themselves have capabilities comparable to human prompt engineers [24]. Therefore, we briefly explore the effects of LLM-generated mutation prompts. The following meta-prompt was given to GPT-4-0613 [1]:

Meta prompt

Now, imagine yourself as a prompt engineer, you give <LLM> a piece of optimization algorithm code, and you want <LLM> to modify it by $x\%$ (where x is a predefined number from 2 to 40, and indicated the code difference between the new one and the old one) exactly to improve the algorithm performance (meaning optimization convergence speed, not code efficiency, for example, by trying different mutation, selection, etc. strategies), what prompt would you give? Please give me at least 3 examples. Do not propose any specific directions or elements to change, since we want to cover the whole algorithm search space.

where <LLM> should be replaced with the specific model name when using this meta-prompt.

4 Experimental Setup

Large Language Models. This study uses two large language models, GPT-4o and GPT-3.5-turbo, which have advanced capabilities and performance in generating meta-heuristic optimization algorithms.

- **GPT-4o-2024-08-06**: this is OpenAI's latest large-scale multi-modal language model, known for its ability to understand and generate complex code.
- **GPT-3.5-turbo-0125**: an earlier version of the GPT model, which lags behind the newer models but still shows good performance when dealing with complex tasks.

We choose these two models due to their popularity and ease of implementation. It would be interesting to extend the experimental setup with additional (open-weight) models in future work.

Choice of Fixed Mutation Rates. Since we expect to achieve dynamic mutation rates sampled from a heavy-tailed power-law distribution, and with a maximum mutation rate of no more than 50% of the distribution, we choose 2%, 5%,

10%, 20%, and 40% as important references to determine how well an LLM is able to mutate the code according to these rates.

Automatically Generated Prompts. Using the meta-prompt of Sect. 3.2, we obtain **prompt 6** to **prompt 11**, where **prompt 6** to **prompt 8** correspond to the model GPT-3.5-turbo and **prompt 9** to **prompt 11** correspond to the GPT-4o model. These prompts are shown below:

Generated prompts for GPT-3.5-turbo

Prompt 6 Adjust the code such that the algorithm's convergence speed is improved, while ensuring that the changes result in an exact difference of $x\%$ compared to the original code. This difference should reflect the modification in functionality, not code style or syntax. Feel free to adjust any part of the algorithm (e.g., initialization, selection, mutation, or other components) to achieve faster convergence while maintaining the specified code difference.

Prompt 7 Modify the optimization algorithm code to improve its performance in terms of convergence speed. The modification should result in a code difference of exactly $x\%$. Ensure that the changes are meaningful to enhance optimization speed without focusing on code efficiency or readability improvements. Explore any strategy within the algorithm to achieve this, but keep the difference precisely at the specified percentage.

Prompt 8 Please enhance the convergence speed of the optimization algorithm given below by modifying it. The modifications should introduce a code difference of precisely $x\%$ compared to the original code. Focus on optimizing the algorithm's behavior rather than its implementation efficiency. You are free to explore any area of the algorithm's logic, but ensure that the total code difference remains exactly at $x\%$ and is geared toward faster convergence.

Generated prompts for GPT-4o

Prompt 9 Here's a piece of code for an optimization algorithm. Please modify it by exactly $x\%$ to improve the algorithm's performance in terms of optimization convergence speed. Focus on introducing meaningful changes that can potentially enhance its effectiveness, such as exploring alternative strategies or approaches across any aspect of the algorithm. Keep the modifications strictly within the specified $x\%$ range for code difference while striving for faster convergence.

Prompt 10 Take this code of an optimization algorithm and adjust it by $x\%$ to improve convergence speed. Make sure the modifications cover a broad spectrum of possible algorithm adjustments, considering changes

across different components without exceeding $x\%$ in code difference. Your changes should aim to improve the algorithm's ability to reach optimal solutions more quickly.

Prompt 11 Please transform this optimization algorithm code by exactly $x\%$ in a way that enhances convergence speed. Keep the code difference precisely at $x\%$, and focus solely on achieving performance improvements through algorithmic adjustments across various elements of the code. Avoid focusing on code efficiency; instead, prioritize exploration of diverse approaches within the allowed modification percentage.

4.1 Experimental Design

In our experiments, LLaMEA is applied to the following scenario: using a $(1+1)$-selection strategy to propose and improve metaheuristic algorithms for solving single-objective continuous optimization problems, see Fig. 2. In each iteration, LLaMEA mutates the parent metaheuristic algorithm to obtain a child algorithm, and only the better algorithm of them can be the new parent for the next iteration. To determine which algorithm is better, the parent and child is tested for their effectiveness with the widely used Black-Box Optimization Benchmarking (BBOB) test set [8] within the IOHexperimenter platform [15], with the mean AOCC as the measure of algorithm performance [20].

The dimensionality is set to 5 for all BBOB problems, the tests cover 3 instances of each problem, and the generated metaheuristic algorithms are repeated 3 times on each problem instance to eliminate experimental randomness and ensure reliable results. The evaluation budget is 10000 for each problem instance.

The experiments consisted of two parts: First, we explore the reliability of the mutation prompts. The second experiment aims to explore the possible enhancements that the dynamic mutation rate brings to LLaMEA. The following is the setup of the first experiment:

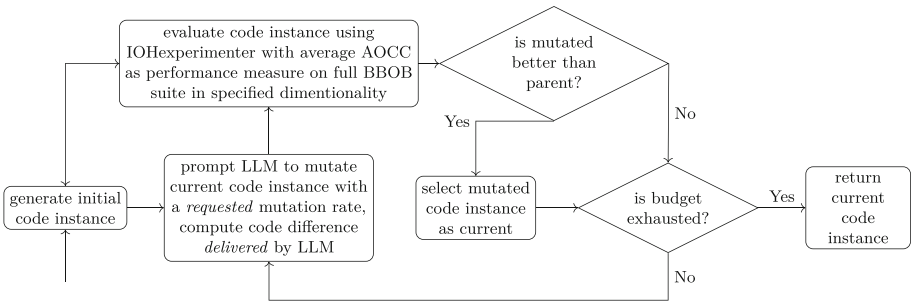


Fig. 2. LLaMEA with controlled mutation via dynamic prompts.

Models GPT-3.5-turbo-0125 and GPT-4o-2024-08-06
Evolutionary strategy (1 + 1)
Requested mutation rates 2%, 5%, 10%, 20% and 40%
Prompts used a total of 11 different prompts, of which 5 are manually constructed to be generic and automatically generated to be model-specific, i.e., both GPT-3.5-turbo and GPT-4o are experimented with 8 prompts.
Number of independent runs the experiment is repeated 3 times for each combination (model, requested mutation rate, prompt).
Code generation budget 100 code instances are generated for each configuration.

After finishing the first experiment, we select the best-performing manually constructed prompts and automatically generated prompts for GPT-3.5-turbo and GPT-4o, respectively, and then experiment the effect of using a dynamic mutation rate on the basis of these prompts in LLaMEA. The setup for this followup experiment is as follows:

Models GPT-3.5-turbo-0125 and GPT-4o-2024-08-06
Evolutionary strategy (1 + 1)
Requested mutation rates dynamic mutation rate, with $\beta = 1.5$ in Eq. 1
Prompts used the best-performing manually constructed prompts and automatically generated prompts for GPT-3.5-turbo and GPT-4o, respectively
Number of independent runs the experiment is repeated 5 times for each combination (model, prompts).
Code generation budget 100 code instances generated for each configuration.

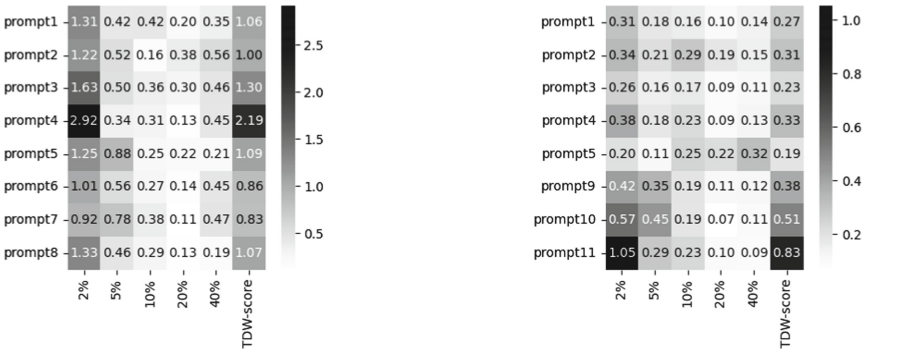
This exhaustive experimental setup allows a comprehensive assessment of the potential benefits of dynamic mutation in automated algorithm generation.

5 Results

5.1 Reliability of Mutation Prompts

In Fig. 3, the MSE and TDW-scores are shown, where x-axis and y-axis represent requested mutation rates and prompts, respectively. For both Figs. 3a and 3b, the first five columns from the left illustrate MSE values, and the last column on the right shows the TDW-score, which is the most important metric for selecting the prompts (the smaller the value, the better the prompt). Thus, we can find that prompt2 is the best manually constructed prompt and prompt7 is the best automatically generated prompt for GPT-3.5-turbo, but for GPT-4o, prompt5 and prompt9 are the best manually constructed prompts and best automatically generated prompts, respectively.

In addition, there is a very noticeable difference for both models: the MSE and TDW-scores of GPT-3.5-turbo are significantly *inferior* to those of GPT-4o. In order to explore the details behind these numbers in depth, we show the distribution of difference of the codes generated with different prompts and requested mutation rates, as well as a scatterplot of the ratio of actual delivered code difference to requested mutation rate for LLaMEA, displayed in Fig. 4.



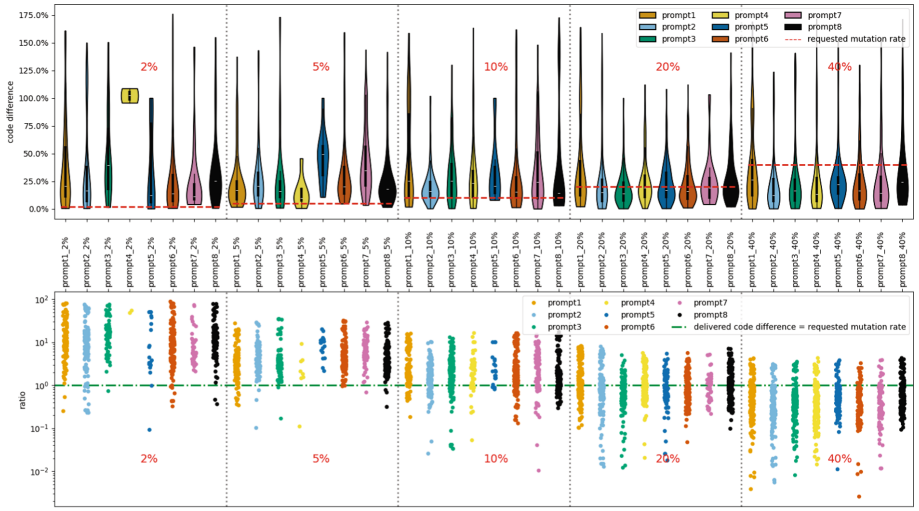
(a) GPT-3.5-turbo (b) GPT-4o

Fig. 3. MSE and TDW-scores of prompts. For both figures, the x-axis is the requested mutation rates, the first five columns from left show the corresponding MSE for the requested mutation rate, and the last column on the right shows the TDW-score. The smaller the value, the better the prompt.

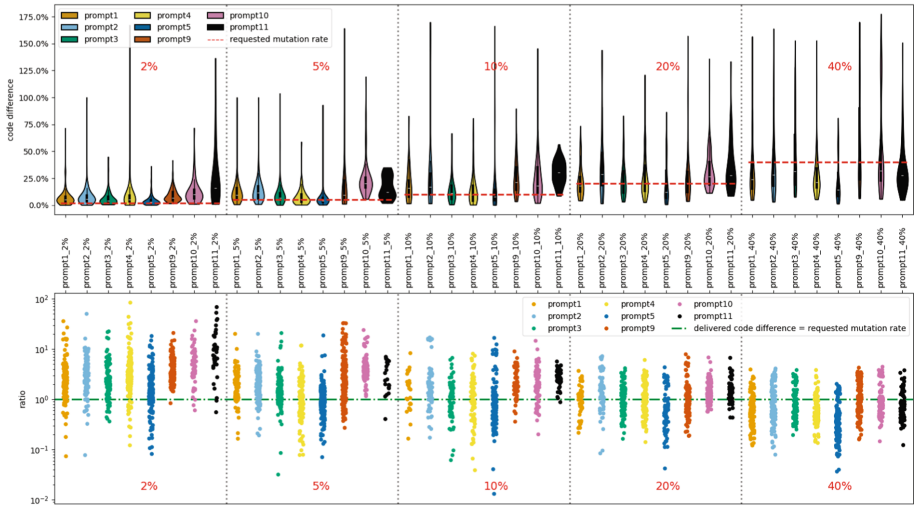
In Figs. 4a and 4b, the red dashed line represents the requested mutation rate, and the green dotted-dashed line indicates that the delivered code difference of the points on this line is equal to the requested mutation rate. Each column of data contains mutated codes from the 100 codes generated in 3 independent runs using LLaMEA and the requested mutation rate in each prompt.

From Fig. 4a, we can see that progressively more complex and accurate artificially constructed prompts do *not* improve the performance of LLaMEA with GPT-3.5-turbo. Moreover, it seems that GPT-3.5-turbo does *not* understand the mutation rate variation well. The code difference of the code generated by GPT-3.5-turbo always stays high regardless of whether the requested mutation rate is high or low, and the distribution is not concentrated.

From Fig. 4b, we find that the distribution of code difference for GPT-4o is more *concentrated* compared to the result of GPT-3.5-turbo, and also *closer* to the requested mutation rates. Therefore, GPT-4o is a significantly better choice than GPT-3.5-turbo for LLaMEA when economic conditions allow (GPT-4o is roughly five times more expensive than the GPT-3.5-turbo). Second, as the artificially constructed prompts become more complex (longer), the delivered code difference gets closer to the mutation rate, that is, the scatterplot gets closer and closer to the green dotted dashed line, especially when the mutation rate is 2%, 5%, and 10%. This is encouraging, and it shows that GPT-4o *understands* the task of mutation prompts, and that prompt engineering is effective for mutation prompts and GPT-4o performs the mutation task quite well.

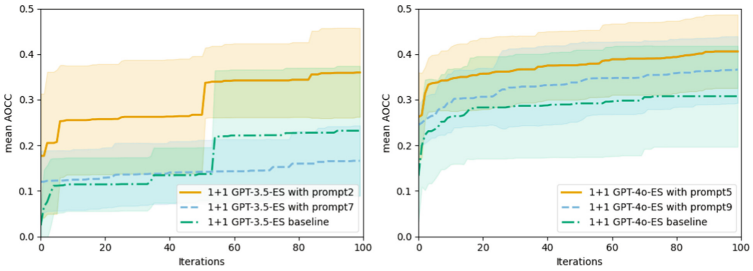


(a) GPT-3.5-turbo



(b) GPT-4o

Fig. 4. The distribution of code difference generated with different prompts and requested mutation rates (top figure for each LLM) and scatterplot of the ratio of actual delivered code difference to requested mutation rate for LLaMEA (bottom figure for each LLM). Colours of distributions and scatter points correspond to different prompts. Results of different mutation rates (shown in red) are separated by vertical gray dotted lines. The red dashed line represents the requested mutation rate, while the green dotted-dashed line shows that the difference in the delivered code for points on this line equals the requested mutation rate. Each data column contains mutated codes from 100 codes generated over 3 runs. (Color figure online)



(a) GPT-3.5-turbo (b) GPT-4o

Fig. 5. Mean convergence curves (best-so-far algorithm scores) over the 5 LLaMEA experiments for each selected prompt and LLM. Shaded areas denote the standard deviation of the best-so-far. Baselines are the raw data from [18]. We found that for GPT-3.5-turbo and GPT-4o, the artificially constructed mutation prompt significantly improves the AOCC, which implies that the quality of the algorithms generated by LLaMEA improves.

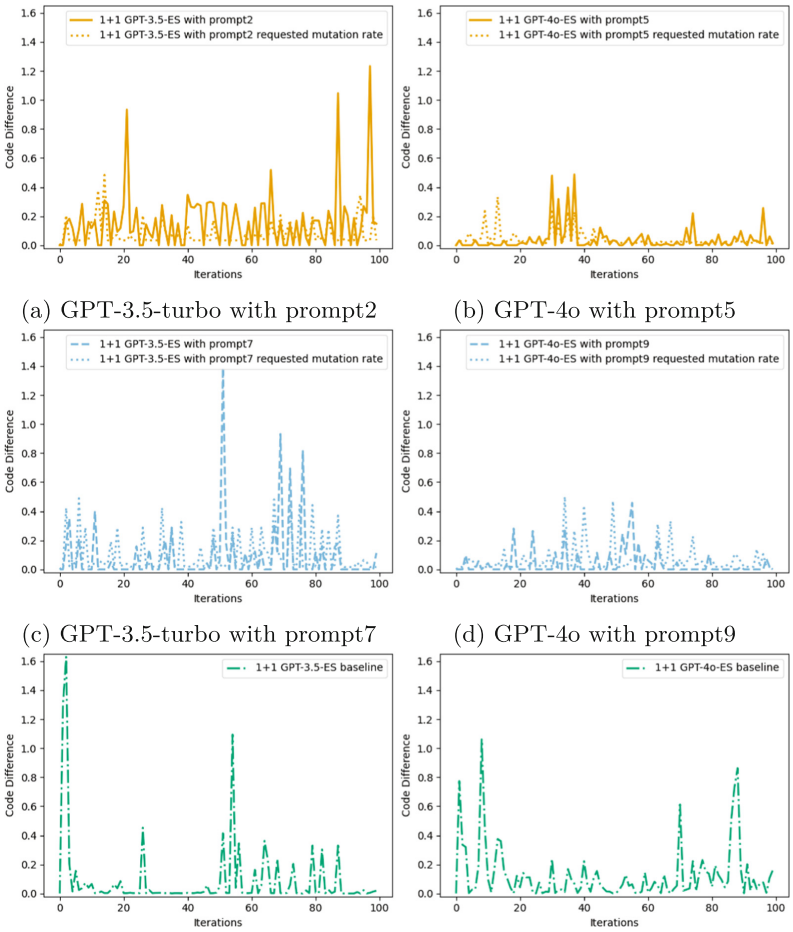
5.2 Influence of Dynamic Mutation Rate

Figure 5 compares the convergence speed of the baselines and LLaMEA with the dynamic mutation rate as introduced in Sect. 3 using the best mutation prompt as identified before. Baselines are raw data directly from [18], which represent the default setting of LLaMEA without dynamic mutation rate. Shaded areas denote the standard deviation of the best-so-far.

From Fig. 5, we can observe that the manually constructed mutation prompt with dynamic mutation rate using GPT-4o improves the performance of LLaMEA with the $(1 + 1)$ selection, but on the other hand, the automatically generated mutation prompt is not much different from the baseline (even slightly worse). Considering that this experiment is based on relatively simple automatic prompt engineering, there is still room for exploring more efficient automatic prompt generation methods to define better mutation prompts.

For the GPT-3.5-turbo model, the prompt used seems to have little effect on the convergence speed, and both are weaker than the baseline (though not significantly). The overall performance of GPT-3.5-turbo is also significantly weaker than that of GPT-4o, which implies that the comprehension capacity of GPT-3.5-turbo becomes a bottleneck in improving the effectiveness of LLaMEA.

Figure 6 shows the change in the code difference of the code generated by LLaMEA after applying the prompts of the dynamic mutation rate. Dynamically required mutation rates are generally small, and compared to GPT-3.5-turbo, GPT-4o is significantly easier to mutate at the required mutation rate. Without specifying a specific value for the mutation rate, both GPT-3.5-turbo and GPT-4o are more likely to carry out extreme mutations, as shown in Figs. 6e, 6f.



(a) GPT-3.5-turbo with prompt2 (b) GPT-4o with prompt5

(c) GPT-3.5-turbo with prompt7 (d) GPT-4o with prompt9

(e) GPT-3.5-turbo baseline (f) GPT-4o baseline

Fig. 6. Example code difference of one run out of five runs for each selected prompt and LLM. Baselines are the raw data from [18]. For Figs. 6a, 6b, 6c, and 6d, the dotted lines represent the requested mutation rate.

6 Discussions and Future Work

In this study, we observe that the design of mutation prompts has a significant impact on the ratio of code mutations applied by the LLM in an algorithm optimization task, and additionally influences the optimization performance. The experimental results show that there is a difference in performance between the manually constructed prompts and the automatically generated prompts, suggesting that the precise design of the prompts is crucial to the optimization effect of the algorithm. For the GPT-3.5-turbo model, the performance is not

satisfactory even when the indication of the mutation rate is more explicit, which may be related to the model's ability to understand the mutation task.

The introduction of dynamic mutation rates can improve the adaptability of the LLaMEA algorithm and lead to better generated meta-heuristic algorithms. By experimental comparison, the LLaMEA framework using dynamic mutation rate has better convergence speed and optimization results than the default prompt without mutation rate when using GPT-4o. This finding supports the effectiveness of using dynamic control strategies in evolutionary algorithms. Despite the experimental success, there are still challenges in precisely controlling the mutation rate. In addition, advanced automatic prompt engineering techniques may require more development and testing in practice to produce and optimize more efficient algorithms.

Future research should consider developing more advanced automated prompt engineering techniques. Research should also be extended to a wider variety and newer versions of local LLMs to assess the scalability and adaptability of mutation strategies and reduce funding consumption.

References

1. Achiam, J., et al.: GPT-4 technical report. arXiv preprint arXiv:2303.08774 (2023)
2. Antipov, D., Doerr, B.: Runtime analysis of a heavy-tailed $(1 + (\lambda, \lambda))$ genetic algorithm on jump functions. In: Bäck, T., et al. (eds.) PPSN 2020. LNCS, vol. 12270, pp. 545–559. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58115-2_38
3. Bäck, T., Schwefel, H.P.: An overview of evolutionary algorithms for parameter optimization. Evol. Comput. **1**(1), 1–23 (1993)
4. Beyer, H.G., Schwefel, H.P.: Evolution strategies-a comprehensive introduction. Natural Comput. **1**, 3–52 (2002)
5. Böttcher, S., Doerr, B., Neumann, F.: Optimal fixed and adaptive mutation rates for the leadingones problem. In: Schaefer, R., Cotta, C., Kołodziej, J., Rudolph, G. (eds.) PPSN 2010, Part I. LNCS, vol. 6238, pp. 1–10. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15844-5_1
6. Chang, Y., et al.: A survey on evaluation of large language models. ACM Trans. Intell. Syst. Technol. **15**(3), 1–45 (2024)
7. Doerr, B., Le, H.P., Makhmara, R., Nguyen, T.D.: Fast genetic algorithms. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 777–784 (2017)
8. Hansen, N., Finck, S., Ros, R., Auger, A.: Real-parameter black-box optimization benchmarking 2009: noiseless functions definitions. Ph.D. thesis, INRIA (2009)
9. Hansen, N., Ostermeier, A.: Completely derandomized self-adaptation in evolution strategies. Evol. Comput. **9**(2), 159–195 (2001)
10. Holland, J.H.: Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. MIT Press, Cambridge (1992)
11. Jansen, T., Wegener, I.: On the analysis of a dynamic evolutionary algorithm. J. Discrete Algorithms **4**(1), 181–199 (2006)
12. Lange, R., Tian, Y., Tang, Y.: Large language models as evolution strategies. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, pp. 579–582 (2024)

13. Liu, F., Tong, X., Yuan, M., Zhang, Q.: Algorithm evolution using large language model. arXiv preprint arXiv:2311.15249 (2023)
14. Liu, F., et al.: Evolution of heuristics: towards efficient automatic algorithm design using large language model. In: Forty-First International Conference on Machine Learning (2024)
15. de Nobel, J., Ye, F., Vermetten, D., Wang, H., Doerr, C., Bäck, T.: IOHexperimenter: benchmarking Platform for Iterative Optimization Heuristics. arXiv e-prints:2111.04077 (2021). https://arxiv.org/abs/2111.04077
16. Ojha, V.K., Abraham, A., Snášel, V.: Metaheuristic design of feedforward neural networks: a review of two decades of research. Eng. Appl. Artif. Intell. **60**, 97–116 (2017)
17. Penas, D.R., González, P., Egea, J.A., Banga, J.R., Doallo, R.: Parallel metaheuristics in computational biology: an asynchronous cooperative enhanced scatter search method. Procedia Comput. Sci. **51**, 630–639 (2015)
18. van Stein, N., Bäck, T.: LLaMEA: a large language model evolutionary algorithm for automatically generating metaheuristics. IEEE Trans. Evol. Comput. 1 (2024). https://doi.org/10.1109/TEVC.2024.3497793
19. van Stein, N., Vermetten, D., Bäck, T.: In-the-loop hyper-parameter optimization for LLM-based automated design of heuristics. arXiv preprint arXiv:2410.16309 (2024)
20. van Stein, N., Vermetten, D., Kononova, A.V., Bäck, T.: Explainable benchmarking for iterative optimization heuristics. arXiv preprint arXiv:2401.17842 (2024)
21. Wu, X., Wu, S.h., Wu, J., Feng, L., Tan, K.C.: Evolutionary computation in the era of large language model: survey and roadmap. arXiv preprint arXiv:2401.10034 (2024)
22. Yang, X.S.: Optimization and metaheuristic algorithms in engineering. Metaheuristics Water Geotech. Transp. Eng. **1**, 23 (2013)
23. Zhao, W.X., et al.: A survey of large language models. arXiv preprint arXiv:2303.18223 (2023)
24. Zhou, Y., et al.: Large language models are human-level prompt engineers. In: NeurIPS 2022 Foundation Models for Decision Making Workshop (2022). https://openreview.net/forum?id=YdqwNaCLCx